

EECS 348 Assignment 3 - GenAI Code Comparison

Author: Samyu Aitha

Date: September 29, 2026

A-B. GenAI Programs and Access

1. Google Gemini. I used Gemini through Google's Gemini web interface in a web browser.
2. Anthropic Claude. I used Claude through Anthropic's Claude web interface in a web browser.

C. Exact Prompt Used for Both GenAIs

Objective: Write a C++ program using Object-Oriented Programming (no standalone functions outside classes, no standard/external heap libraries) to implement a list-based MaxHeap priority queue for prioritizing a CEO's emails.

Email Priority Rules

1. Category Priority (Highest to Lowest):

- `Boss` > `Subordinate` > `Peer` > `ImportantPerson` > `OtherPerson`

2. Date Tie-Breaker:

- If two emails share the same category, the **newest** date (`MM-DD-YYYY`) takes higher priority.

Input File Commands (`input.txt` or passed via `argv[1]`)

- `EMAIL <Category>,<Subject>,<Date>` — Parse and insert email into the priority queue.
- `COUNT` — Output: `There are <N> emails to read.`
- `NEXT` — Display the top priority email without removing it: Plaintext

```
Next email:
Sender: <Category>
Subject: <Subject>
Date: <Date>
```

- **READ** — Remove the top priority email from the queue without printing anything.

Edge Cases & Execution Requirements

- Dynamic memory management for node pointer links (binary tree structure).
- Handle **NEXT** or **READ** gracefully when the queue is empty.
- Support consecutive **NEXT** operations (display same top email) and consecutive **READ** operations (pop without printing).

D. Complete Raw Generated Code

Gemini Raw Output

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
// Forward declaration
class Email;
// Date representation to assist with chronological comparisons
class Date {
private:
    int month;
    int day;
    int year;
public:
    Date() : month(0), day(0), year(0) {}
    Date(const std::string& dateStr) {
        // Expected format: MM-DD-YYYY
        std::stringstream ss(dateStr);
        std::string token;
        if (std::getline(ss, token, '-')) month = std::stoi(token);
        if (std::getline(ss, token, '-')) day = std::stoi(token);
        if (std::getline(ss, token, '-')) year = std::stoi(token);
    }
}
```

```

    bool operator<(const Date& other) const {
        if (year != other.year) return year < other.year;
        if (month != other.month) return month < other.month;
        return day < other.day;
    }
    bool operator>(const Date& other) const {
        return other < *this;
    }
    bool operator==(const Date& other) const {
        return year == other.year && month == other.month && day == other.day;
    }
};
// Represents an individual Email
class Email {
private:
    std::string categoryStr;
    std::string subject;
    std::string dateStr;
    Date date;
    int categoryPriority; // Higher number = higher priority
    int evaluateCategoryPriority(const std::string& cat) const {
        if (cat == "Boss") return 5;
        if (cat == "Subordinate") return 4;
        if (cat == "Peer") return 3;
        if (cat == "ImportantPerson") return 2;
        if (cat == "OtherPerson") return 1;
        return 0;
    }
}

```

```

public:
    Email() : categoryPriority(0) {}
    Email(const std::string& cat, const std::string& sub, const std::string& dt)
        : categoryStr(cat), subject(sub), dateStr(dt), date(dt) {
        categoryPriority = evaluateCategoryPriority(cat);
    }
    std::string getCategory() const { return categoryStr; }
    std::string getSubject() const { return subject; }
    std::string getDateString() const { return dateStr; }
    // Priority ordering rules:
    // 1. Sender Category (Boss > Subordinate > Peer > ImportantPerson > OtherPerson)
    // 2. Newer Date > Older Date
    bool operator>(const Email& other) const {
        if (categoryPriority != other.categoryPriority) {
            return categoryPriority > other.categoryPriority;
        }
        return date > other.date; // Newest email first
    }
    bool operator<(const Email& other) const {
        if (categoryPriority != other.categoryPriority) {

```

```

        return categoryPriority < other.categoryPriority;
    }
    return date < other.date;
}
};
// Node structure for dynamic list-based MaxHeap implementation
class HeapNode {
public:
    Email data;
    HeapNode* parent;
    HeapNode* left;
    HeapNode* right;
    HeapNode(const Email& email)
        : data(email), parent(nullptr), left(nullptr), right(nullptr) {}
};
// Custom List-based MaxHeap implementation
class MaxHeap {
private:
    HeapNode* root;
    int heapSize;
    // Helper method to obtain a node at a specific 1-based index using binary representation pathing
    HeapNode* getNodeAtIndex(int index) const {
        if (index <= 0 || index > heapSize) return nullptr;
        if (index == 1) return root;
        // Collect bit path from root to index
        std::vector<int> path;
        int temp = index;
        while (temp > 1) {
            path.push_back(temp % 2); // 0 = left, 1 = right
            temp /= 2;
        }
        HeapNode* current = root;
        for (int i = path.size() - 1; i >= 0; --i) {
            if (path[i] == 0) {
                current = current->left;
            } else {
                current = current->right;
            }
        }
        return current;
    }
    void heapifyUp(HeapNode* node) {
        while (node->parent != nullptr && node->data > node->parent->data) {
            std::swap(node->data, node->parent->data);
            node = node->parent;
        }
    }
    void heapifyDown(HeapNode* node) {
        while (node != nullptr) {
            HeapNode* largest = node;
            if (node->left != nullptr && node->left->data > largest->data) {

```

```

        largest = node->left;
    }
    if (node->right != nullptr && node->right->data > largest->data) {
        largest = node->right;
    }
    if (largest != node) {
        std::swap(node->data, largest->data);
        node = largest;
    } else {
        break;
    }
}
}
void destroyTree(HeapNode* node) {

```

```

    if (node != nullptr) {
        destroyTree(node->left);
        destroyTree(node->right);
        delete node;
    }
}
public:
    MaxHeap() : root(nullptr), heapSize(0) {}
    ~MaxHeap() {
        destroyTree(root);
    }
    bool empty() const {
        return heapSize == 0;
    }
    int size() const {
        return heapSize;
    }
    void push(const Email& email) {
        heapSize++;
        HeapNode* newNode = new HeapNode(email);
        if (root == nullptr) {
            root = newNode;
            return;
        }
        // Attach new node at position `heapSize`
        HeapNode* parentNode = getNodeAtIndex(heapSize / 2);
        newNode->parent = parentNode;
        if (heapSize % 2 == 0) {
            parentNode->left = newNode;
        } else {
            parentNode->right = newNode;
        }
        heapifyUp(newNode);
    }
}

```

```

    Email top() const {
        if (empty()) {
            throw std::runtime_error("Heap is empty.");
        }
        return root->data;
    }
    void pop() {
        if (empty()) return;
        if (heapSize == 1) {
            delete root;
            root = nullptr;
            heapSize = 0;
            return;
        }
        HeapNode* lastNode = getNodeAtIndex(heapSize);
        root->data = lastNode->data;
        // Detach last node from parent
        HeapNode* parentNode = lastNode->parent;
        if (parentNode->left == lastNode) {
            parentNode->left = nullptr;
        } else {
            parentNode->right = nullptr;
        }
        delete lastNode;
        heapSize--;
        heapifyDown(root);
    }
};
// Priority Queue wrapper utilizing list-based MaxHeap
class PriorityQueue {
private:
    MaxHeap heap;
public:
    void enqueue(const Email& email) {
        heap.push(email);
    }
    void dequeue() {
        if (!heap.empty()) {
            heap.pop();
        }
    }
    Email peek() const {
        return heap.top();
    }
    bool isEmpty() const {
        return heap.empty();
    }
    int getCount() const {

```

```

        return heap.size();
    }
};
// Command Processor encapsulating test execution
class CommandProcessor {
private:
    PriorityQueue emailQueue;
    void processLine(const std::string& line) {
        if (line.empty()) return;
        if (line.rfind("EMAIL ", 0) == 0) {
            std::string payload = line.substr(6);
            std::stringstream ss(payload);
            std::string category, subject, date;
            if (std::getline(ss, category, ',') &&
                std::getline(ss, subject, ',') &&
                std::getline(ss, date, ',')) {
                Email email(category, subject, date);
                emailQueue.enqueue(email);
            }
        } else if (line == "COUNT") {
            std::cout << "There are " << emailQueue.getCount() << " emails to read." << s
td::endl;
        } else if (line == "NEXT") {
            if (!emailQueue.isEmpty()) {
                Email topEmail = emailQueue.peek();
                std::cout << "Next email:" << std::endl;
                std::cout << "Sender: " << topEmail.getCategory() << std::endl;
                std::cout << "Subject: " << topEmail.getSubject() << std::endl;
                std::cout << "Date: " << topEmail.getDateString() << std::endl;
            } else {
                std::cout << "No emails to display." << std::endl;
            }
        } else if (line == "READ") {
            if (!emailQueue.isEmpty()) {
                emailQueue.dequeue();
            }
        }
    }
public:
    void processFile(const std::string& filename) {
        std::ifstream inFile(filename);
        if (!inFile.is_open()) {
            std::cerr << "Error opening file: " << filename << std::endl;
            return;
        }
        std::string line;
        while (std::getline(inFile, line)) {
            // Trim trailing carriage return characters (\r) for cross-platform compati
lity
            if (!line.empty() && line.back() == '\r') {
                line.pop_back();
            }
        }
    }
};

```

```

        processLine(line);
    }
    inFile.close();
}
};
int main(int argc, char* argv[]) {
    std::string filename = "input.txt";
    if (argc > 1) {
        filename = argv[1];
    }
    CommandProcessor processor;
    processor.processFile(filename);
    return 0;
}

```

Claude Raw Output

```

// ceo_email_queue.cpp
// Prioritizes a CEO's inbox using a list-based MaxHeap built from scratch.
// No heap libraries are used (no std::priority_queue, std::make_heap, etc.).
//
// Build: g++ -std=c++17 -Wall -o ceo_email_queue ceo_email_queue.cpp
// Run: ./ceo_email_queue test.txt
// (or run with no argument and type the file name when prompted)
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <stdexcept>
#include <cctype>
// -----
// Email: holds one message and knows how to compare its priority to another.
// -----
class Email {
public:
    Email(const std::string& sender, const std::string& subject,
          const std::string& date, long arrivalOrder)
        : sender_(sender), subject_(subject), date_(date),
          arrivalOrder_(arrivalOrder),
          categoryRank_(rankForCategory(sender)),
          dateKey_(keyForDate(date)) {}
    // True if this email should be read before "other".
    // 1) Higher sender category wins (Boss > Subordinate > Peer > ImportantPerson > OtherPerson)
    // 2) Same category: newer date wins
    // 3) Same category and date: the one that arrived later in the file wins
    bool hasHigherPriorityThan(const Email& other) const {
        if (categoryRank_ != other.categoryRank_) return categoryRank_ > other.categoryRank_;
    }
};

```

```

        if (dateKey_ != other.dateKey_) return dateKey_ > other.dateKey_;
        return arrivalOrder_ > other.arrivalOrder_;
    }
    void display(std::ostream& out) const {
        out << "Next email:\n"
            << "\tSender: " << sender_ << "\n"
            << "\tSubject: " << subject_ << "\n"
            << "\tDate: " << date_ << "\n";
    }
private:
    std::string sender_;
    std::string subject_;
    std::string date_;
    long arrivalOrder_;
    int categoryRank_;
    long dateKey_;
    static int rankForCategory(const std::string& category) {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        if (category == "OtherPerson") return 1;
        return 0; // unknown category: read after everything else
    }
    // Converts MM-DD-YYYY into YYYYMMDD so dates compare as plain numbers.
    static long keyForDate(const std::string& date) {
        if (date.size() != 10 || date[2] != '-' || date[5] != '-') return 0;
        try {
            long month = std::stol(date.substr(0, 2));
            long day = std::stol(date.substr(3, 2));
            long year = std::stol(date.substr(6, 4));
            return year * 10000 + month * 100 + day;
        } catch (...) {
            return 0;
        }
    }
};
// -----
// MaxHeap: list-based (array-backed) binary max-heap of Email objects.
// Index math for node i: parent (i-1)/2, left child 2i+1, right child 2i+2.
// -----
class MaxHeap {
public:
    bool isEmpty() const { return items_.empty(); }
    size_t size() const { return items_.size(); }
    void insert(const Email& email) {
        items_.push_back(email);
        siftUp(items_.size() - 1);
    }
    const Email& peekMax() const {
        if (isEmpty()) throw std::out_of_range("peekMax on empty heap");
        return items_[0];
    }
};

```

```

}
void removeMax() {
    if (isEmpty()) throw std::out_of_range("removeMax on empty heap");
    items_[0] = items_.back();

```

```

        items_.pop_back();
        if (!isEmpty()) siftDown(0);
    }
private:
    std::vector<Email> items_;
    static size_t parent(size_t i) { return (i - 1) / 2; }
    static size_t left(size_t i) { return 2 * i + 1; }
    static size_t right(size_t i) { return 2 * i + 2; }
    void swapItems(size_t a, size_t b) {
        Email temp = items_[a];
        items_[a] = items_[b];
        items_[b] = temp;
    }
    void siftUp(size_t i) {
        while (i > 0 && items_[i].hasHigherPriorityThan(items_[parent(i)])) {
            swapItems(i, parent(i));
            i = parent(i);
        }
    }
    void siftDown(size_t i) {
        size_t n = items_.size();
        while (true) {
            size_t largest = i;
            size_t l = left(i), r = right(i);
            if (l < n && items_[l].hasHigherPriorityThan(items_[largest])) largest = l;
            if (r < n && items_[r].hasHigherPriorityThan(items_[largest])) largest = r;
            if (largest == i) break;
            swapItems(i, largest);
            i = largest;
        }
    }
};
// -----
// CEOInbox: the priority queue the CEO works from, built on MaxHeap.
// -----
class CEOInbox {
public:
    explicit CEOInbox(std::ostream& out) : out_(out), nextArrival_(0) {}
    void addEmail(const std::string& sender, const std::string& subject,
                  const std::string& date) {
        heap_.insert(Email(sender, subject, date, nextArrival_++));
    }
    // NEXT: show the top email without removing it (repeated NEXTs show the same one).
    void showNext() const {

```

```

        if (heap_.isEmpty()) {
            out_ << "No emails to read.\n";
            return;
        }
        heap_.peekMax().display(out_);
    }
    // READ: remove the top email whether or not it was displayed.
    void readTop() {
        if (heap_.isEmpty()) return; // nothing to read; quietly ignore
        heap_.removeMax();
    }
    void showCount() const {
        out_ << "There are " << heap_.size() << " emails to read.\n";
    }
private:
    std::ostream& out_;
    MaxHeap heap_;
    long nextArrival_;
};
// -----
// CommandProcessor: reads the test file and dispatches each command.
// -----
class CommandProcessor {
public:
    explicit CommandProcessor(CEOInbox& inbox) : inbox_(inbox) {}
    bool processFile(const std::string& fileName) {
        std::ifstream in(fileName);
        if (!in) {
            std::cerr << "Error: could not open file \"" << fileName << "\"\n";
            return false;
        }
        std::string line;
        while (std::getline(in, line)) processLine(line);
        return true;
    }
    void processLine(const std::string& rawLine) {
        std::string line = trim(rawLine);
        if (line.empty()) return;
        size_t space = line.find(' ');
        std::string command = (space == std::string::npos) ? line : line.substr(0, space);
        std::string rest = (space == std::string::npos) ? "" : trim(line.substr(space + 1));

```

```

        if (command == "EMAIL") handleEmail(rest);
        else if (command == "NEXT") inbox_.showNext();
        else if (command == "READ") inbox_.readTop();
        else if (command == "COUNT") inbox_.showCount();
        else std::cerr << "Warning: unknown command \"" << command << "\" skipped\n";
    }
}

```

```

private:
    CEOInbox& inbox_;
    // Fields: <sender category>,<subject>,<date>
    void handleEmail(const std::string& fields) {
        size_t firstComma = fields.find(',');
        size_t lastComma = fields.rfind(',');
        if (firstComma == std::string::npos || firstComma == lastComma) {
            std::cerr << "Warning: malformed EMAIL line skipped: " << fields << "\n";
            return;
        }
        std::string sender = trim(fields.substr(0, firstComma));
        std::string subject = trim(fields.substr(firstComma + 1, lastComma - firstComma -
1));
        std::string date = trim(fields.substr(lastComma + 1));
        inbox_.addEmail(sender, subject, date);
    }
    // Strips spaces, tabs and Windows '\r' from both ends.
    static std::string trim(const std::string& s) {
        size_t start = 0, end = s.size();
        while (start < end && std::isspace(static_cast<unsigned char>(s[start]))) ++start;
        while (end > start && std::isspace(static_cast<unsigned char>(s[end - 1]))) --end;
        return s.substr(start, end - start);
    }
};
// -----
int main(int argc, char* argv[]) {
    std::string fileName;
    if (argc > 1) {
        fileName = argv[1];
    } else {
        std::cout << "Enter test file name: ";
        std::getline(std::cin, fileName);
    }
    CEOInbox inbox(std::cout);
    CommandProcessor processor(inbox);
    return processor.processFile(fileName) ? 0 : 1;
}

```

E. Code Analysis

Correctness: Gemini

I found that Gemini correctly models `Email`, `Date`, `MaxHeap`, `PriorityQueue`, and `CommandProcessor` as objects. It follows the required sender ranking and uses the newer date when two emails have the same category. Its `NEXT` command looks at

the top email without removing it, while `READ` removes the highest-priority email, so repeated `NEXT` and `READ` commands work as expected. It also follows the required Sender/Subject/Date format. One issue I noticed is that it prints an extra message, "No emails to display.", when `NEXT` is used on an empty queue. Since the assignment does not specify that message, a strict grader could mark the output wrong. The date parser also assumes the date is numeric and does not fully validate the format.

Correctness: Claude

Claude also handles the five sender categories, newest-date priority, repeated `NEXT` commands, and `READ` commands correctly. I liked that its parser trims whitespace and splits an `EMAIL` command using the first and last comma. However, I noticed two unnecessary choices in the generated code. First, the display method adds tab characters before Sender, Subject, and Date even though the sample output does not include tabs. Second, it adds an arrival-order tie-breaker even though the assignment only gives rules for category and date. Both choices could cause a mismatch with a strict grader.

Execution Time

Claude's array-backed `MaxHeap` takes $O(\log n)$ time for insertion and removal because `siftUp` and `siftDown` move through at most the height of the heap. `NEXT` /peek and `COUNT` both take $O(1)$ time. Creating an `Email` is also $O(1)$ because the date has a fixed format. Gemini's pointer-based heap still has $O(\log n)$ heapify operations, but its insertion and removal also call `getNodeAtIndex`, which builds and follows a root-to-node path. Therefore, both programs have the same asymptotic running time, but Gemini does more pointer-based work than Claude's direct index arithmetic.

Space Complexity

Both programs store n emails, so they use $O(n)$ main storage. Claude keeps the heap in one vector, which gives each heap operation $O(1)$ auxiliary space. Gemini creates one `HeapNode` per email, and each node stores three pointers. Its `getNodeAtIndex` method also creates a temporary path vector with $O(\log n)$ space. This gives Gemini more memory overhead and requires a recursive destructor to clean up all of the nodes.

Maintainability

I think Claude's version is easier to maintain because the `MaxHeap` is shorter, uses the standard binary-heap index formulas, and avoids manual `new / delete` operations. The code also separates responsibilities between `Email`, `MaxHeap`, `CEOInbox`, and `CommandProcessor`. Gemini is still object-oriented, but it has more parts to manage, including a separate `Date` class, linked heap nodes, parent/child pointers, a `PriorityQueue` wrapper, and recursive cleanup. In my opinion, those extra parts create more opportunities for pointer bugs. Claude's main weaknesses are the unnecessary arrival-order rule and the small output-format difference.

F. Selected Baseline and Justification

I choose the Claude program as my baseline. Both programs have $O(\log n)$ insertion and removal and $O(n)$ total heap storage, but Claude's vector representation has less structural overhead. It does not need parent, left, or right pointers, separate dynamic allocations for each node, a recursive destructor, or a temporary $O(\log n)$ path vector. Its direct index calculations also make the heap logic shorter and easier for me to verify. The main problems I needed to fix were small: remove the tabs from the `NEXT` output, remove the extra arrival-order tie-breaker, and make empty `READ` and `NEXT` operations safe without printing unspecified text.

G. Improvements Made to the Selected Code

Correctness Improvements

I changed the `NEXT` output so that it matches the sample format exactly: "Next email:", followed by Sender, Subject, and Date with no tab prefix. I removed the arrival-order field and its third tie-breaker so that priority follows only the two rules in the assignment: sender category first and newest date second. I also made `removeMax` return safely when the heap is empty, which allows repeated `READ` commands. For an empty queue, `NEXT` now returns without printing extra text. The `EMAIL` parser still trims whitespace and uses the first and last comma, which is safe because the assignment says subjects will not contain commas.

Execution-Time Improvements

I kept Claude's direct vector index calculations in the final version. `insert` remains $O(\log n)$, `READ` / `removeMax` remains $O(\log n)$, and `NEXT` and `COUNT` remain $O(1)$. The category and date priorities are calculated when each `Email` is created, so the program does not need to repeatedly parse dates during heap comparisons.

Space-Complexity Improvements

The final heap uses $O(n)$ vector storage and $O(1)$ auxiliary space during `siftUp` and `siftDown`. Removing `arrivalOrder` also removes one unnecessary field from every `Email` object. Overall, this design avoids Gemini's three pointers per email and its temporary $O(\log n)$ path vector.

Maintainability Improvements

I simplified `Email` so it stores only the information required by the assignment. I kept the class responsibilities clear: `Email` handles priority logic, `MaxHeap` handles heap operations, `CEOInbox` provides priority-queue behavior, and `CommandProcessor` parses the input commands. I also added a complete prolog, source and collaborator attribution, block comments, and line-level comments because the assignment rubric specifically grades documentation and attribution.

Verification

The improved C++ program was compiled with `g++ -std=c++17 -Wall -Wextra -pedantic` and executed against the sample test file. Its output matched the sample output provided in the assignment instructions.

Expected Sample Output

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.